

drone-video-link: Đường truyền hình ảnh độ trễ thấp và cổng điều khiển cho drone

Báo cáo kỹ thuật (hướng Đi làm) — giải thích từ đầu cho người mới

Học viên: Phong — Định hướng: Embedded Software Engineer

I. Tổng quan báo cáo

Báo cáo này mô tả **drone-video-link**, một bản mẫu (prototype) phần *ground-station link* — đường liên kết giữa drone và trạm mặt đất — cho một payload camera gắn trên drone. Hệ thống gồm hai nửa chạy song song:

- **Đường truyền hình ảnh (video downlink):** lấy hình từ camera, nén thành H.264, đóng gói RTP rồi gửi qua mạng UDP xuống trạm mặt đất, với mục tiêu **độ trễ thật thấp**.
- **Cổng MAVLink (telemetry & control gateway):** nhận dữ liệu bay (độ cao, pin, tư thế...) từ bộ điều khiển bay và gửi lệnh (arm, cất cánh, hạ cánh) lên, có kiểm tra phản hồi và có cơ chế an toàn khi mất liên lạc.

Toàn bộ được xây dựng và *đo đạc* trên Embedded Linux, mô phỏng đúng ngăn xếp phần mềm dùng trên máy tính đồng hành (companion computer) Jetson Xavier / Orin NX của một airframe thật, nhưng chạy trọn vẹn trên một chiếc laptop: nguồn video được giả lập bằng **videotestsrc**, còn bộ điều khiển bay được giả lập bằng **ArduPilot SITL**. Không cần drone thật.

Đối tượng đọc

Báo cáo được viết cho **người mới hoàn toàn**. Mỗi khái niệm — codec, RTP, jitter buffer, MAVLink... — đều được giải thích lại từ số không kèm một ví dụ hoặc phép so sánh đời thường, trước khi đưa ra số liệu. Bạn không cần biết trước gì về video streaming hay drone để đọc hiểu.

Vì sao bài toán này đáng làm. Truyền video thời gian thực từ một vật thể bay là một bài toán kỹ thuật cổ điển nhưng đầy cạm bẫy: mạng không dây hay mất gói, drone di chuyển nhanh nên độ trễ vài trăm mili-giây là không chấp nhận được, và bộ giải mã video có thể *giấu* lỗi khiến phần mềm tưởng mọi thứ vẫn ổn trong khi người lái nhìn thấy hình vỡ. Project này không chỉ dựng cho video chạy được, mà **đo** chính xác từng đánh đổi đó bằng công cụ thật.

Kết quả nổi bật (tất cả đều đo được, không phỏng đoán).

- Độ trễ pipeline giảm từ **2288 ms** (cấu hình mặc định) xuống **7,8 ms** sau khi tinh chỉnh — và nguyên nhân chính *không* phải B-frame như nhiều tài liệu nói.
- Ở mức mất 20% gói, **474 gói** biến mất nhưng ứng dụng vẫn nhận 148/150 khung hình và cờ báo lỗi của bộ giải mã vẫn bằng **0** — bằng chứng cho thấy phải đo mất gói ở *tầng RTP*.
- Một gói bị mất làm hỏng hình **cho tới keyframe kế tiếp**: 29/29 đọt hỏng trên bốn ảnh thử đều kết thúc đúng tại một keyframe.
- Cổng MAVLink phát hiện **đứt liên lạc trong 3 giây** (kiểm thử 7/7), và lệnh `takeoff` 10 arm rồi leo tới 10 m, lặp lại được trên ArduPilot SITL thật.

Mục lục

I.	Tổng quan báo cáo	1
II.	Định nghĩa bài toán	5
II.1.	Bối cảnh	5
II.2.	Bài toán 1 — Đường truyền hình ảnh	5
II.3.	Bài toán 2 — Cổng MAVLink	5
III.	Các khái niệm nền tảng	6
III.1.	Bức tranh tổng thể: một khung hình đi từ camera tới màn hình	6
III.2.	Nén video H.264: I-frame, P-frame, GOP, slice	6
III.3.	Vì sao video dùng UDP chứ không TCP, và RTP thêm gì	7
III.4.	GStreamer: pipeline, element, pad, caps	7
III.5.	Jitter buffer: đánh đổi trung tâm của cả project	8
III.6.	MAVLink: heartbeat, message, command, SITL	8
IV.	Kiến trúc hệ thống	9
IV.1.	Luồng Video	9
IV.2.	Luồng MAVLink	10
V.	Thành phần 1 — Đường truyền video H.264/RTP	10
VI.	Thực nghiệm 1 — Đo độ trễ pipeline	10
VI.1.	Thủ phạm thật trong bộ mã hoá — và vì sao <i>không</i> phải B-frame	11
VII.	Thực nghiệm 2 — Mất gói và lan truyền lỗi	12
VII.1.	Cơ chế lan truyền lỗi	12
VII.2.	Dụng cụ đo nào nói thật, dụng cụ nào nói dối	13
VII.3.	Cái giá thật của <code>latency=0</code>	14
VII.4.	Keyframe hay P-frame: mất slice nào tệ hơn?	14
VII.5.	Câu trả lời “sách giáo khoa” là sai	15
VIII.	Thành phần 2 — Cổng MAVLink (telemetry & control)	15
VIII.1.	Watchdog — logic an toàn bay	15
VIII.2.	Bẫy đơn vị — MAVLink gửi số nguyên đã nhân tỉ lệ	16

VIII.3. Lệnh được xác nhận, và đúng thứ tự	16
IX. Thành phần 3 — Dashboard trực quan thời gian thực	17
X. Công cụ và công nghệ sử dụng	17
XI. Cấu trúc mã nguồn và hướng dẫn chạy	18
XII. Phân tích lỗi và hạn chế	19
XII.1. Giới hạn của thí nghiệm	19
XII.2. Những điều đã làm sai rồi sửa	19
XIII. Kết luận và hướng phát triển	20
XIV. Tài liệu tham khảo	21
Phụ lục	22

II. Định nghĩa bài toán

Phần này chuyển vấn đề thực tế thành hai bài toán kỹ thuật cụ thể, để người đọc biết hệ thống nhận gì, tạo ra gì và được đánh giá bằng tiêu chí nào.

II.1. Bối cảnh

Một drone quay phim mang theo hai luồng thông tin cần đưa xuống mặt đất: (1) **hình ảnh** để người điều khiển nhìn thấy camera đang thấy gì, và (2) **dữ liệu bay** (telemetry) như độ cao, tốc độ, pin, tư thế. Chiều ngược lại, mặt đất cần gửi **lệnh** lên drone. Cả ba đều đi qua một liên kết vô tuyến không hoàn hảo: có độ trễ, có mất gói, đôi khi đứt hẳn.

II.2. Bài toán 1 — Đường truyền hình ảnh

- **Đầu vào:** luồng hình thô từ camera (ở đây: 1280×720 , 30 khung hình/giây).
- **Đầu ra:** luồng hình hiển thị được ở trạm mặt đất, kèm các *số đo*: độ trễ, số khung/giây, số gói mất.
- **Loại bài toán:** kỹ thuật truyền thông đa phương tiện thời gian thực (real-time multimedia streaming), không phải học máy.
- **Tiêu chí thành công:** độ trễ pipeline ở mức mili-giây (không phải giây); khi mạng xấu, hệ thống *đo được* mức độ hỏng thay vì im lặng; mọi con số đều tái lập được bằng script trong repo.

II.3. Bài toán 2 — Cổng MAVLink

- **Đầu vào:** các thông điệp MAVLink từ bộ điều khiển bay (heartbeat, vị trí, tư thế, trạng thái pin...).
- **Đầu ra:** nhật ký telemetry đã quy đổi ra đơn vị SI (CSV/JSONL); trạng thái liên kết (còn sống / mất); và kết quả của các lệnh arm / cất cánh / hạ cánh.
- **Loại bài toán:** lập trình giao thức nhị phân + logic an toàn (state machine, watchdog).
- **Tiêu chí thành công:** phát hiện mất liên lạc trong đúng cửa sổ failsafe; mọi lệnh đều được *xác nhận* bằng phản hồi của bộ điều khiển bay; chạy được end-to-end với firmware thật (SITL).

Điểm mấu chốt

Trọng tâm của project không phải *làm cho nó chạy* — video streaming và gửi lệnh MAVLink là việc nhiều người làm được. Trọng tâm là **đo và giải thích các đánh đổi**: độ trễ đổi lấy độ mượt, keyframe đổi lấy khả năng hồi phục, tin cậy đổi lấy tốc độ. Đó là điều một kỹ sư nhúng thật sự làm hằng ngày.

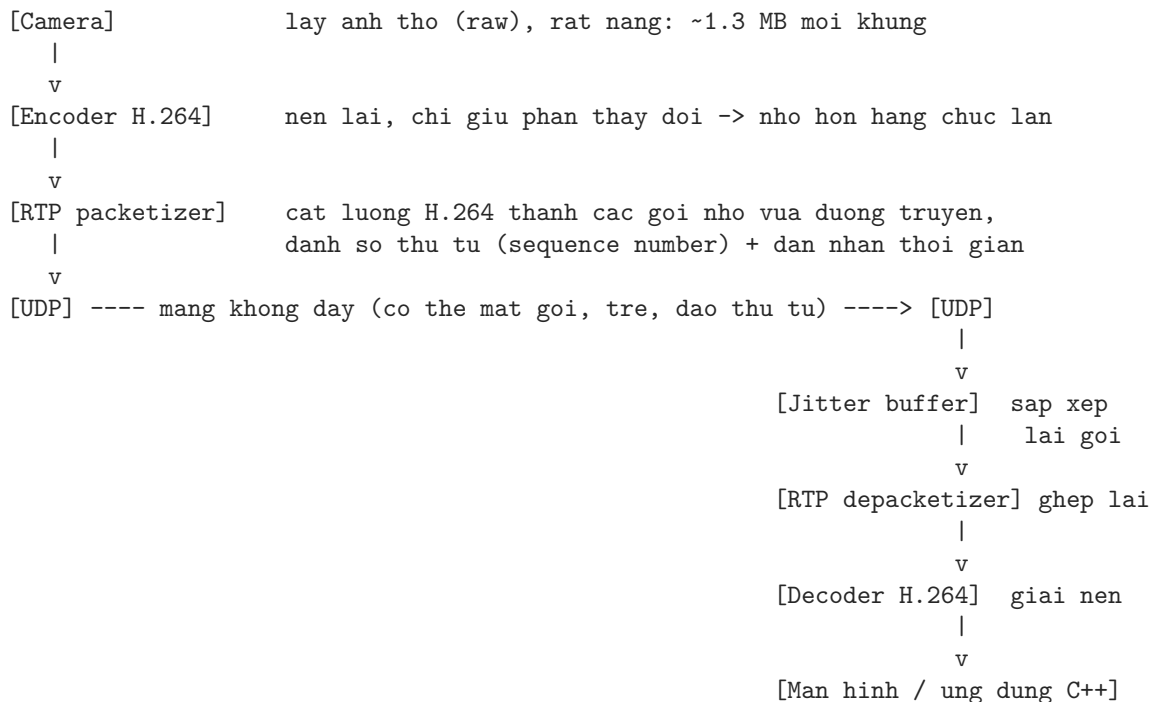
III. Các khái niệm nền tảng

Đây là phần dài nhất và quan trọng nhất với người mới. Nắm được sáu tiểu mục dưới đây là đủ để hiểu toàn bộ phần còn lại của báo cáo.

III.1. Bức tranh tổng thể: một khung hình đi từ camera tới màn hình

Hãy hình dung một **dây chuyền lắp ráp**. Một khung hình đi qua từng trạm, mỗi trạm làm một việc, rồi chuyển sang trạm sau:

Đường đi của một khung hình



Hai nửa của dây chuyền nằm ở hai nơi khác nhau trong đời thật: nửa bên trái (tới UDP) chạy trên **máy tính đồng hành** gắn trên drone (Jetson); nửa bên phải chạy trên **trạm mặt đất**. *Ranh giới mạng* nằm đúng ở chỗ hai khối UDP. Trong project, cả hai chạy trên cùng một máy qua 127.0.0.1 (loopback) để dễ đo.

III.2. Nén video H.264: I-frame, P-frame, GOP, slice

Vì sao phải nén? Một khung hình 1280×720 thô nặng khoảng 1,3 MB. Ở 30 khung/giây, đó là gần 40 MB mỗi giây — không đường truyền không dây nào chịu nổi. Bộ mã hoá (encoder) H.264 nén luồng này xuống còn vài trăm KB/giây bằng một ý tưởng đơn giản: **giữa hai khung liên tiếp, phần lớn điểm ảnh không đổi**.

- **I-frame** (còn gọi *keyframe*): một tấm ảnh hoàn chỉnh, tự giải mã được, không cần tham chiếu khung nào khác. To.
- **P-frame**: chỉ lưu **phần khác biệt** so với khung trước. Rất nhỏ.
- **GOP** (Group of Pictures): một I-frame và toàn bộ P-frame phụ thuộc vào nó. Tham số `key-int-max=30` nghĩa là *tối đa 30 khung thì chèn một keyframe* — ở 30 fps là đúng **một keyframe mỗi giây**.
- **Slice**: mỗi khung có thể bị cắt thành nhiều *dải ngang* độc lập. Khi bật đa luồng theo kiểu `tune=zerolatency`, đo được **11 slice mỗi khung**. Điều này quyết định: mất một gói làm hỏng một *dải*, không phải cả khung hình.

I-frame to hơn P-frame bao nhiêu lần? Đây là tính chất của *nội dung*, không phải của H.264. Đo bằng `gop-stats.sh` trên luồng thật: thanh màu SMPTE 2,8 lần, quả bóng trên nền đen 3,2 lần, zone-plate động 8,4 lần, pinwheel **15,0 lần**. Càng ít chuyển động thì P-frame càng rộng, tỉ số càng lớn.

III.3. Vì sao video dùng UDP chứ không TCP, và RTP thêm gì

TCP bảo đảm không mất gói và đúng thứ tự bằng cách *truyền lại* gói mất và giữ mọi gói sau đó lại chờ. Với video thời gian thực, điều này là tai họa: một gói của khung #5 mà đến sau khi khung #10 đã hiển thị thì *đã vô dụng*; tệ hơn, việc chờ nó làm mọi khung sau bị treo — video đứng hình rồi tua nhanh.

Video thời gian thực thà **mất dữ liệu** còn hơn **trễ dữ liệu**. Đó đúng là hợp đồng của **UDP**: gửi đi, không hứa hẹn gì. Nhưng UDP trần trụi quá — không thứ tự, không phát hiện mất gói, không nhãn thời gian. Nên người ta đặt thêm một header nhỏ tên **RTP** (Real-time Transport Protocol) vào bên trong mỗi gói UDP, mang theo:

- **Sequence number** — để bên nhận biết gói nào mất, gói nào đến sai thứ tự.
- **Timestamp** — thời điểm khung được lấy mẫu, để phát lại đúng nhịp.
- **Payload type** — cho biết đây là H.264 (số 96) hay codec khác.

Câu chốt dễ nhớ: “*UDP nói: tôi không hứa gì. RTP nói: nhưng tôi cho bạn biết cái gì đã mất.*” Chính jitter buffer (mục sau) dùng các thông tin RTP này để làm việc.

III.4. GStreamer: pipeline, element, pad, caps

GStreamer là khung phần mềm đa phương tiện dùng để *ghép* cái dây chuyền ở trên. Bốn khái niệm:

- **Element** — một trạm trong dây chuyền (ví dụ `x264enc` là bộ mã hoá).

- **Pad** — cổng của element: *sink pad* là đầu vào, *src pad* là đầu ra (hơi ngược trực giác: “src” là chỗ dữ liệu *đi ra*).
- **Caps** (capabilities) — mô tả dữ liệu qua một pad: định dạng, độ phân giải, framerate. Ví dụ `video/x-raw,width=1280,height=720`.
- **Pipeline** — cả dây chuyền cộng với bus báo lỗi và clock đồng bộ.

Ký hiệu ! trong lệnh `gst-launch` nghĩa là “nối src pad của cái bên trái vào sink pad của cái bên phải”. Khi nối, hai element *tự thương lượng* định dạng chung; không có định dạng chung thì lỗi `not-negotiated`.

Một cạm bẫy kinh điển cho người mới

Bên nhận `udpsrc` lấy gói từ mạng chỉ thấy một đồng byte, không biết đó là H.264 hay rác, nên **phải khai caps thủ công**, kèm số payload type. Sender khai `pt=96` thì receiver phải khai đúng 96. Lệnh số $\Rightarrow$ không nhận được gì và **không có thông báo lỗi** — pipeline chạy im lặng, màn hình đen. Đây là lỗi “màn hình đen” nổi tiếng khi mới học.

III.5. Jitter buffer: đánh đổi trung tâm của cả project

Gói UDP đến **không đều nhịp** (hiện tượng gọi là *jitter*) và có thể **sai thứ tự**. `rtpjitterbuffer` giữ gói lại một chút, sắp xếp theo sequence number, rồi nhả ra đều đặn.

- `latency=200` (mặc định): giữ gói 200 ms. Mượt, chịu được jitter, nhưng **cộng thêm 200 ms vào độ trễ**.
- `latency=0`: nhả ngay. Trễ thấp nhất, nhưng gói đến muộn hoặc lệch thứ tự sẽ bị **vứt bỏ** $\Rightarrow$ giật hình.

Đây là **đánh đổi trung tâm**: độ trễ đổi lấy độ mượt. Không có đáp án đúng tuyệt đối, chỉ có đáp án phù hợp với tình huống. Drone FPV chọn `latency=0` (cần phản xạ nhanh); xem phim trên mạng chọn buffer vài giây (cần mượt). Ở mục VI. và VII. sẽ thấy cái giá thật của lựa chọn này, đo được.

III.6. MAVLink: heartbeat, message, command, SITL

MAVLink là một *giao thức nhị phân* để phần mềm nói chuyện với drone. Nó không phải một thư viện — `pymavlink` chỉ là một hiện thực của nó. Header rất nhỏ (khoảng 12 byte), có checksum, chạy trên bất kỳ đường truyền nào (UART, UDP, radio), nên code viết một lần dùng được ở mọi nơi.

- **Heartbeat**: mọi thành phần phát “nhịp tim” ở **1 Hz**. Quy ước ngành: **mất 3 nhịp liên tiếp (≈ 3 giây) coi như đứt link**, và drone phải kích hoạt failsafe (thường là bay về điểm cất cánh).

- **Message** (thông điệp): dữ liệu phát liên tục một chiều, không ai xác nhận — ví dụ tư thế, vị trí. Cứ chảy về đều đều.
- **Command** (lệnh): gửi lên và bộ điều khiển bay **phải trả lời** bằng một **COMMAND_ACK**. Bắn lệnh rồi cho là xong (fire-and-forget) là sai — FC có thể từ chối, và ta cần biết vì sao.
- **SITL** (Software In The Loop): chính firmware ArduPilot biên dịch cho máy tính x86 thay vì cho drone. Nó chạy đầy đủ bộ lọc trạng thái (EKF), bộ điều khiển, kiểm tra trước khi arm và mô phỏng vật lý. Nghĩa là *giao thức, thời gian, hành vi từ chối lệnh — tất cả đều thật*. Đây là quy trình chuẩn của ngành, không phải “đồ chơi vì thiếu drone”.

IV. Kiến trúc hệ thống

Hệ thống gồm hai luồng dữ liệu độc lập, chia sẻ chung một câu chuyện “đường liên kết cho drone”. Chúng không cần tích hợp code với nhau.

IV.1. Luồng Video

Luong du lieu - Video

```
[videotestsrc]    nguồn giả lập, thay cho camera CSI (I420, 1280x720, 30fps)
|
[timeoverlay]    in timestamp lên hình để đo latency bằng mắt
|
[videoconvert]   đổi color format cho khớp encoder
|
[x264enc]         nén H.264, tune=zerolatency (lookahead=0 + sliced threads)
|
[rtpv264pay]     cat thành gói RTP, thêm seq number + timestamp
|
[udpsink] ==> UDP 127.0.0.1:5000 ==> [udpsrc]    (tc netem chen loss/jitter)
|
|               [rtpjitterbuffer]    latency=0 để giảm trễ
|               |
|               [rtpv264depay]       ghép RTP lại thành H.264
|               |
|               [avdec_h264]         giải nén
|               |
[appsink]        ==> receiver.cpp: dem frame, đọc PTS, in
stats
```

Mọi thứ bên trái **udpsink** là companion computer trên drone; mọi thứ bên phải **udpsrc** là trạm mặt đất. **appsink** là điểm mấu chốt: nó *đưa từng khung hình vào tay code C++* thay vì chỉ vẽ lên màn hình — đây cũng là chỗ ngoài đời cần AI inference (đẩy khung sang TensorRT vẽ bounding box).

IV.2. Luồng MAVLink

Luồng dữ liệu - MAVLink

```
[ArduPilot SITL] == UDP 14550 ==> [gateway.py]
(gia lap FC)      MAVLink v2      |
^                  +--> watchdog: mat heartbeat 3s -> LINK LOST
|                  +--> telemetry.csv / telemetry.jsonl
+---- COMMAND_LONG -----+ +--> MQTT: topic drone/telemetry [tuy chon]
|      (arm/takeoff/land)   |
+---- COMMAND_ACK <-----+ gateway doc ACK, khong "ban roi quen"
```

MAVLink là giao thức **hai chiều bất đối xứng**: telemetry được FC phát liên tục (streaming), còn lệnh theo mô hình gửi-chờ-xác nhận (request/ack). Code được chia đúng theo đường nối đó.

V. Thành phần 1 — Đường truyền video H.264/RTP

Nửa video gồm một *sender* (script `gst-launch`) và một *receiver* C++ dùng GStreamer C API. Receiver không chỉ hiển thị mà còn *đếm*: số khung, số khung/giây, và — quan trọng nhất — số gói mất đọc từ jitter buffer.

Vì sao receiver là C++ và appsink làm gì

`autovideosink` vẽ khung lên màn hình rồi thôi. `appsink` thì trao từng khung cho code: callback `new_sample` được gọi mỗi khi có khung mới, code lấy con trỏ vào vùng điểm ảnh, đọc timestamp, rồi **bắt buộc giải phóng** bộ nhớ (`unref`). Quên một lần `unref` là rò rỉ bộ nhớ (memory leak), RAM tăng dần cho tới khi tiến trình bị hệ điều hành kết liễu. Đây chính là loại lỗi bộ nhớ mà kỹ sư nhúng phải gỡ trong đời thật.

Một chi tiết đồng bộ hoá đáng nói: callback `new_sample` chạy trên *streaming thread*, còn vòng lặp in thống kê chạy mỗi giây trên một luồng khác. Vì thế các bộ đếm trong `receiver.cpp` dùng `std::atomic` để hai luồng đọc/ghi an toàn.

VI. Thực nghiệm 1 — Đo độ trễ pipeline

Cách đo. Không tự viết code đo. GStreamer có sẵn bộ theo dõi (tracer) bật bằng một biến môi trường, in ra độ trễ của cả pipeline và của từng element để quy trách nhiệm cho đúng thủ phạm:

Bật tracer đo độ trễ của GStreamer

```
GST_TRACERS="latency(flags=pipeline+element)" gst-launch-1.0 ...
```

Mọi số dưới đây là **trung vị** của 150 khung (bỏ 15 khung đầu làm nóng máy), trên WSL2 Ubuntu 22.04, 16 lõi, GStreamer 1.20.3, luồng 1280×720@30, 2 Mbit/s.

#	Cấu hình	Pipeline	x264enc	Jitter buffer
1	mặc định (latency=200, sync=true)	2288 ms	2067 ms	218 ms
2	+ tune=zerolatency speed-preset=ultrafast	223 ms	4,2 ms	216 ms
3	+ rtpjitterbuffer latency=0	23 ms	4,2 ms	15,5 ms
4	+ sink sync=false	7,8 ms	4,1 ms	0,8 ms

Bảng 1: Bốn cấu hình, độ trễ giảm dần từ 2288 ms xuống 7,8 ms.

Đọc từng bước:

- **1→2 (encoder, −2064 ms):** thắng lợi lớn nhất, đến *toàn bộ* từ bộ mã hoá. Chi tiết ở dưới.
- **2→3 (jitter buffer, −200 ms):** đúng bằng latency đã hạ. Đây là cái giá phải trả *trên mọi gói*, dù mạng có xấu hay không.
- **3→4 (sink, −15 ms):** tắt việc sink chờ đúng nhãn thời gian mới hiển thị. Vì không có audio để đồng bộ, chờ đó là trễ vô ích.

VI.1. Thủ phạm thật trong bộ mã hoá — và vì sao *không* phải B-frame

Tách riêng bộ mã hoá (giữ jitter buffer = 0 và tắt sync), chỉ đổi dòng encoder:

Cấu hình encoder	x264enc (trung vị)	quy ra khung @30fps
x264enc (mặc định)	2067 ms	62
+ sliced-threads=true	1340 ms	40
+ rc-lookahead=0	734 ms	22
+ speed-preset=ultrafast	734 ms	22
+ tune=zerolatency	5,8 ms	0,2
+ tune=zerolatency ultrafast	4,2 ms	0,1

Bảng 2: Hai nguồn trễ độc lập trong encoder cộng lại: 1340 + 734 ≈ 2067 ms.

Nhiều bài viết nói tune=zerolatency nhanh vì nó *tắt B-frame* (loại khung phải chờ khung tương lai). **Điều này sai với element này:** x264enc của GStreamer mặc định bframes=0 — không có B-frame nào để mà tắt. Kiểm tra một dòng: `gst-inspect-1.0 x264enc | grep bframes`. Hai thủ phạm thật là:

1. **Lookahead** (rc-lookahead, mặc định 40): encoder giữ lại 40 khung để bộ điều khiển bitrate “nhìn trước” cảnh sắp tới rồi mới quyết định tiêu bao nhiêu bit cho khung hiện tại. Cái giá là **đúng 40 khung độ trễ** — đo được 39,4 khung khi ép chạy 1 luồng.

2. **Đa luồng theo khung:** mỗi luồng mã hoá một khung khác nhau cùng lúc, nên khung N chưa ra được cho tới khi luồng giữ nó xong việc. Đo được: 4 luồng $\rightarrow$ 10 khung trễ, 8 luồng $\rightarrow$ 14, tự động trên 16 lõi $\rightarrow$ 28. **tune=zerolatency** chuyển sang *sliced threads* (chia một khung cho nhiều luồng) nên độ trễ này biến mất.

Bài học đo đạc

speed-preset=ultrafast một mình chỉ hạ được xuống 734 ms và **kẹt ở đó**, vì nó chỉ đưa lookahead về 0 chứ không dùng tới đa luồng theo khung. Phải có **tune=zerolatency** mới xử lý cả hai. Đây là kiểu chi tiết mà chỉ khi *đo tách bạch* mới thấy được, còn đọc tài liệu thì dễ kết luận sai.

VII. Thực nghiệm 2 — Mất gói và lan truyền lỗi

Câu hỏi phỏng vấn kinh điển: *vì sao mất **một** gói UDP lại làm hỏng hình **cả giây**?* Project trả lời bằng thí nghiệm, dùng tc netem để cố ý làm mạng xấu.

tc netem — vừa giả lập mạng xấu vừa là dụng cụ đo

tc (traffic control) là công cụ của nhân Linux để định hình lưu lượng; netem là module giả lập mạng. Ví dụ netem loss 2% delay 20ms 5ms = mất 2% gói, trễ 20 ms, jitter ± 5 ms. Quan trọng: tc -s qdisc show còn *in ra số gói đã bị đánh rơi* — đây là **sự thật của nhân**, dùng để đối chiếu khi một bộ đếm khác nói dối.

VII.1. Cơ chế lan truyền lỗi

Với GOP 30 khung (I-frame ở #0 và #30):

1. Khung #5 mất một slice $\rightarrow$ hình vỡ ở dải đó.
2. Khung #6 là P-frame: nó nói “giống #5, thêm chút thay đổi”. Nhưng #5 đã sai $\rightarrow$ #6 sai theo.
3. Lỗi **lan truyền** qua #7, #8... vì P-frame chép nguyên khối sai từ khung trước.
4. Đến khung #30 — một I-frame mới, mã hoá không tham chiếu ai $\rightarrow$ hình **hồi phục hoàn toàn**.

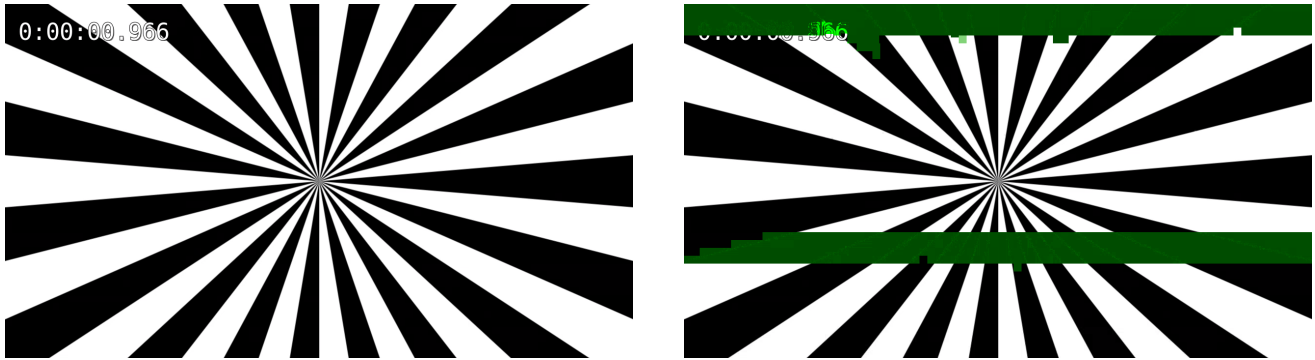
Đây là điều được *đo*, không suy luận. So từng điểm ảnh với luồng gốc: **29/29 đợt hỏng** (trên bốn ảnh thử) kết thúc *đúng* tại một keyframe; khi tự chọn gói để xoá thay vì để netem bắn ngẫu nhiên thì là **36/36**. Sai lệch *không phai dần* mà phẳng lì suốt cả GOP rồi biến mất:

Sai lệch trung bình so với lượng góc (thang 0..255)

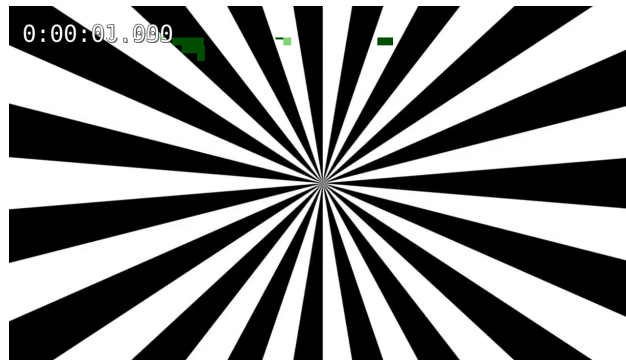
```

khung 12..29 : ~22.6   (phang, khong giam)
khung 30      : 0.336  <- IDR moi, hoi phuc
khung 60      : 0.000  <- IDR sach, trung khit tung bit

```



Hình 1: Trái: khung #29 như được gửi đi. Phải: khung #29 như nhận được — một gói của keyframe GOP này đã mất từ 29 khung trước, hai slice (hai dải ngang) không bao giờ tới, và mọi khung kể từ đó chép cái lỗi đó tiếp về sau.



Hình 2: Khung #30 — keyframe kế tiếp. Mã hoá từ số không nên không nợ khung trước điều gì; các dải hồng biến mất, hình hồi phục.

VII.2. Dụng cụ đo nào nói thật, dụng cụ nào nói dối

Đây là phát hiện hữu ích nhất của cả thí nghiệm. Bộ giải mã `avdec_h264` che giấu slice bị mất trong im lặng: nó dựng lại một khung trông **giống hệt** một khung tốt dưới con mắt của phần mềm.

Hệ quả thiết kế quan trọng nhất của project

Nếu trạm mặt đất muốn biết video có đáng tin hay không, nó **phải đếm ở tầng RTP** (thuộc tính `stats` → `num-lost` của `rtpjitterbuffer`). Không có gì phía *sau* bộ giải mã nói cho bạn biết. Một đèn “video khoẻ mạnh” dựng trên số khung hay cờ giải mã sẽ báo xanh trong khi người lái nhìn thấy hình vỡ nát. Chính vì thế `receiver.cpp` được mở rộng

netem loss	RTP nhận	RTP mất	khung giải mã	DISCONT	CORRUPT
0%	2362	0	150	0	0
2%	2310	45	150	0	0
5%	2238	118	150	0	0
10%	2132	233	150	0	0
20%	1884	474	148	0	0

Bảng 3: Ở 20% mất gói, 474 gói không bao giờ tới, ứng dụng vẫn nhận 148/150 khung và *không* có lỗi nào được bật.

để phát `num-lost` mỗi giây, và dashboard theo dõi đúng con số đó.

Một lưu ý về sự trung thực của dụng cụ: `num-lost` *trùng khít* với `tc` khi mạng chỉ mất gói (86 = 86), nhưng **vọt lên** khi mạng đảo thứ tự gói — nó đếm *số lần phát hiện khoảng trống*, không phải *số gói không tới*. Một bộ đếm chưa được đối chiếu với một bộ đếm độc lập thì chưa phải là số liệu.

VII.3. Cái giá thật của `latency=0`

Ở mục VI., `latency=0` tiết kiệm 200 ms. Đây là hoá đơn. Cho mạng jitter ± 5 ms mà **không** mất gói nào:

netem delay 20ms 5ms, loss 0%

```
tc danh roi      : 0          (mang khong mat goi nao)
num-late         : 3495       (goi den "muon" theo jitter buffer)
khung giai ma duoc : 67 / 300
```

Nhân không mất gì cả, nhưng jitter buffer sâu bằng 0 vớt bỏ ba phần tư luồng đã đến nơi đầy đủ, vì mỗi gói bị trễ ngẫu nhiên làm đảo thứ tự và buffer *không có bộ nhớ* để chứa gói đến sớm. Bật `latency=200` lên thì giải mã được 270–300/300. Nói cách khác: **200 ms độ trễ mua lấy khoảng cách giữa 67 và 300 khung**. Câu chốt: *`latency=0` không chịu nổi một mạng; nó chỉ chịu nổi một sợi dây.*

VII.4. Keyframe hay P-frame: mất slice nào tệ hơn?

Tệ hơn **5,8 lần** nếu mất slice của keyframe — nhưng không vì lý do người ta hay nói. Xoá đúng một slice NAL rồi giải mã (không dùng netem, để loại bỏ ngẫu nhiên):

- Thời gian hỏng không phụ thuộc loại khung — nó chỉ là *khoảng cách tới keyframe kế tiếp*. IDR ở khung 30 tốn 30 khung; P ở khung 15 tốn 15 khung; mất slice ở khung 29 chỉ hỏng *một* khung.
- Biên độ mới phụ thuộc loại khung, và quyết định ngay tại khung trùng. Che một dải *intra* (của keyframe) là việc khó — decoder không có gì để chép từ đó. Toàn bộ 5,8 lần

Nạn nhân	Số khung hỏng	Sai lệch tại khung trúng	Khung tệ nhất
một slice của IDR (khung 30)	30..59 (mọi lần)	10,69	24,31
một slice của P (khung 15)	15..29 (mọi lần)	1,85	2,65

Bảng 4: Thang 0..255. Cả hai điều rút ra đều ngược trực giác.

đã có mặt ngay ở khung đầu tiên; tích lũy suốt phần còn lại của GOP chỉ cộng thêm 3%.

Hệ quả thiết kế: giảm `key-int-max` $\Rightarrow$ hồi phục nhanh hơn nhưng tổn bitrate hơn (I-frame to). Đây là một đánh đổi thật mà kỹ sư video chọn hằng ngày.

VII.5. Câu trả lời “sách giáo khoa” là sai

Lời giải phổ biến: một gói mất rơi vào một khung ngẫu nhiên đều, nên trung bình hỏng nửa GOP — 15,5 khung. Nhưng gói mất không rơi đều theo *khung*, nó rơi đều theo *gói*, mà keyframe chiếm **15% số gói** (to gấp 15 lần P-frame). Tính theo phân bố gói thật: **17,6 khung**, không phải 15,5. Đo thực tế còn cao hơn, trung bình **24,1 khung** (≈ 803 ms) cho mỗi GOP bị trúng, vì ở tỉ lệ mất gói thật một GOP hỏng thường bị trúng nhiều hơn một lần.

VIII. Thành phần 2 — Cổng MAVLink (telemetry & control)

VIII.1. Watchdog — logic an toàn bay

Mất heartbeat của trạm mặt đất là thứ kích hoạt failsafe của máy bay thật. Ngưỡng mặc định của ArduPilot khoảng 3 giây (ba nhịp 1 Hz bị lỡ), nên gateway báo mất link trên cùng ngân sách đó:

monitor phát hiện đứt link

```
hb#1    STABILIZE disarmed alt 0.0m 12.6V
hb#2    STABILIZE disarmed alt 0.0m 12.6V
hb#3    STABILIZE disarmed alt 0.0m 12.6V
*** LINK LOST *** no heartbeat for 3.0s
```

`watchdog.py` là một máy trạng thái nhỏ, kích hoạt theo cạnh (edge-triggered): callback “mất” / “phục hồi” chỉ bắn *đúng một lần* mỗi lần chuyển trạng thái. Nó nhận một *clock* *tiêm được* nên ngưỡng 3 giây được kiểm thử **chính xác mà không cần ngủ 3 giây thật** — 7/7 test, gồm cả trường hợp biên (khoảng lặng *đúng bằng* ngưỡng thì chưa coi là mất).

VIII.2. Bấy đơn vị — MAVLink gửi số nguyên đã nhân tỉ lệ

Để tiết kiệm băng thông, MAVLink không gửi số thực mà gửi số nguyên đã scale. Đổi sai là log ra “rác hợp lý” mà không hề báo lỗi:

Trường	Trên đường truyền	Đổi ra
vĩ độ / kinh độ (GLOBAL_POSITION_INT)	degE7 (int32)	$\div 10^7 \rightarrow$ độ
độ cao	mm (int32)	$\div 1000 \rightarrow$ mét
roll / pitch / yaw (ATTITUDE)	radian	$\times 180/\pi \rightarrow$ độ
điện áp pin (SYS_STATUS)	mV	$\div 1000 \rightarrow$ volt

Bảng 5: `alt` là độ cao so với mực nước biển; `relative_alt` là so với điểm cất cánh. Kiểm tra “cất cánh 10 m” phải nhìn `relative_alt`.

Parser gộp mọi thông điệp vào *một* snapshot đang tiến hoá, nên một dòng log mô tả trọn vẹn trạng thái máy bay dù vị trí, tư thế, pin đến từ các thông điệp khác nhau.

VIII.3. Lệnh được xác nhận, và đúng thứ tự

Trình tự arm $\rightarrow$ takeoff của ArduPilot, dừng ngay ở bước đầu tiên bị từ chối:

Trình tu takeoff

```
cho GPS 3D fix + EKF -> GUIDED -> arm (thu lai qua cua so pre-arm) -> NAV_TAKEOFF
```

Ở mode GUIDED, FC mới nghe lệnh từ phần mềm. Arm bị *từ chối* nếu EKF chưa hội tụ hoặc chưa có GPS — đây là *tính năng an toàn*, không phải bug. Trên SITL thật, `takeoff 10` arm và leo tới 10 m, lặp lại được qua nhiều lần khởi động lạnh.

Vì sao kiểm thử với SITL thật lại quan trọng

`mock_fc.py` (FC giả) và gateway chung một tác giả, một cách hiểu spec, nên một hiểu lầm chung sẽ qua được cả hai — không phải kiểm chứng độc lập. SITL bắt được **hai lỗi** mà mock giấu: (1) ArduPilot *không* stream telemetry nếu chưa được yêu cầu (`REQUEST_DATA_STREAM`); (2) bit sức khoẻ cảm biến xanh *trước* khi thật sự arm được, nên arm một lần là thua cuộc đua. Cả hai đã sửa. Đây là minh chứng cho giá trị của việc kiểm thử trên một hiện thực độc lập.

IX. Thành phần 3 — Dashboard trực quan thời gian thực

Một trang web đơn hiển thị *cả hai* nửa của liên kết cùng lúc: kênh video (fps, gói mất) và kênh MAVLink (tư thế, độ cao, pin, mode, trạng thái link). Nó đọc *đúng* dữ liệu mà phần còn lại của repo sinh ra — dòng `[stats]` của receiver C++ và `telemetry.jsonl` của gateway — nên không có hiện thực thứ hai bị lệch.

- **Máy chủ (server.py):** HTTP thuần thư viện chuẩn, đẩy sự kiện xuống trình duyệt bằng **Server-Sent Events (SSE)** — một chiều, trình duyệt tự kết nối lại, không cần WebSocket.
- **Video nhúng trong dashboard:** một pipeline thứ hai giải mã luồng RTP, chuyển thành **MJPEG** (chuỗi ảnh JPEG nối tiếp qua HTTP `multipart/x-mixed-replace`) và nhúng vào thẻ `<img>`. Người xem thấy *điểm ảnh thật đang chuyển động*, và chúng *vỡ ra* khi bơm mất gói. Đo được: khung MJPEG 480×270 , khoảng 30 khung/giây (91 khung JPEG trong 3 giây).
- **Chống tranh chấp cổng:** sender dùng một **tee** để nhân đôi luồng RTP ra hai cổng UDP (5000 cho receiver thống kê, 5001 cho pipeline MJPEG), nên hai bên đọc cùng luồng mà không giành nhau một cổng.

Sự trung thực trên kênh video của dashboard

Đèn mất gói trên dashboard là Gói mất · RTP, đọc trực tiếp từ `num-lost`. Ngay cạnh nó là cờ **corrupt** của bộ giải mã, và nó **luôn bằng 0 một cách có chủ đích** — đúng như thực nghiệm 2 đã chứng minh. Dashboard cố tình hiển thị cả hai để cho thấy: chỉ có bộ đếm tăng RTP mới thật sự động đậy khi link xấu.

X. Công cụ và công nghệ sử dụng

Ảnh xạ sang phần cứng thật (Jetson Orin NX)

Trên board thật, `videotestsrc` → `nvarguscamerasrc` (camera CSI), `x264enc` → `nvv4l2h264enc` (mã hoá bằng phần cứng NVENC), `avdec_h264` → `nvv4l2decoder`. Nhưng `rtpH264pay` / `udpsink` / `rtpjitterbuffer` / receiver C++ **giữ nguyên không đổi**. Học pipeline trên PC là học đúng thứ cần học — kiến trúc không đổi, chỉ hai element mã hoá/giải mã được thay bằng bản chạy trên phần cứng.

Công nghệ	Vai trò trong project	Vì sao
GStreamer	dựng cả pipeline gửi/nhận video	chuẩn công nghiệp, chạy được trên Jetson
x264enc	mã hoá H.264 phần mềm	bản mẫu trên PC; đổi sang NVENC trên Jetson
C++17 + CMake	receiver dùng GStreamer C API	hiệu năng, quản lý bộ nhớ tường minh
tc netem	giả lập mạng xấu và đo gói rơi	sự thật của nhân Linux
GST_TRACERS	đo độ trễ từng element	công cụ kỹ sư GStreamer thật dùng
pymavlink	nói giao thức MAVLink	hiện thực tham chiếu, phổ biến
ArduPilot SITL	bộ điều khiển bay giả lập	firmware thật, kiểm chứng độc lập
Python (stdlib) + SSE	máy chủ dashboard	không phụ thuộc ngoài, đẩy real-time
MJPEG	nhúng video vào trang web	đơn giản, không cần plugin
MQTT (tùy chọn)	đẩy telemetry lên broker	tích hợp hệ thống lớn hơn

Bảng 6: Mỗi công nghệ đi kèm lý do lựa chọn, không chỉ liệt kê tên.

XI. Cấu trúc mã nguồn và hướng dẫn chạy

Cau trúc thu mục

```
drone-video-link/
|-- README.md          mat tien tieng Anh cho nha tuyen dung
|-- docs/              so tay ky thuat tieng Viet
|   |-- 04-CONCEPTS.md  giai thich moi khai niem tu dau
|   |-- ...
|-- video/             nua Video (Ngay 1)
|   |-- src/receiver.cpp  app C++ dung GStreamer C API + appsink
|   |-- scripts/         sender.sh, netem.sh, measure-latency.sh, ...
|   |-- results/         latency.md, packet-loss.md, img/*.png <- so lieu do
|   |-- CMakeLists.txt
|-- mavlink/           nua MAVLink (Ngay 2)
|   |-- gateway/         connection, watchdog, telemetry, commands, cli
|   |-- scripts/         run-sitl.sh, mock_fc.py
|   |-- tests/test_watchdog.py  7 test failsafe
|-- dashboard/         trang web truc quan thoi gian thuc
|   |-- server.py  index.html  demo_flight.py  run.sh  stop.sh  loss.sh
```

Chạy lại từng phần:

Các câu lệnh hướng dẫn

```
# 1) Do do tre pipeline
cd video && cmake -B build && cmake --build build
./scripts/measure-latency.sh 150
```

```
# 2) Thi nghiệm mất gói (cần quyền root cho tc netem)
sudo bash scripts/packet-loss.sh

# 3) Cong MAVLink: một của số chạy FC (SITL hoặc mock), một của số chạy gateway
python3 scripts/mock_fc.py
python3 -m gateway.cli takeoff 10

# 4) Dashboard trực quan: mở http://localhost:8090/
wsl -d Ubuntu-22.04 bash dashboard/run.sh
```

Nửa video *không* cần huấn luyện lại mô hình — nó không phải project học máy. Người đọc chỉ cần build receiver một lần rồi chạy các script đo. Mọi số liệu trong báo cáo đều tái lập được bằng đúng các lệnh trên.

XII. Phân tích lỗi và hạn chế

Một báo cáo kỹ thuật trung thực phải nói rõ giới hạn của chính nó. Dưới đây là những điều project *chưa* làm, và những điều tác giả đã *làm sai rồi sửa* — vì sai rồi hiểu vì sao sai là kết quả có giá trị nhất.

XII.1. Giới hạn của thí nghiệm

- **netem sinh lỗi độc lập từng gói.** Sóng vô tuyến thật mất gói theo *chùm* (burst): mất 20 gói liên tiếp khác hẳn mất 20 gói rải rác. Toàn bộ kết quả mất gói ở đây giả định lỗi độc lập, vì đó là thứ netem loss X% tạo ra. Phải nói thẳng điều này nếu bị hỏi.
- **Chưa có radio thật.** Nửa MAVLink được kiểm chứng trên SITL và mock, đều qua loopback. Độ trễ và mất gói của một radio telemetry thật chưa được thử nghiệm ở nửa này (nửa video thì có đo mất gói, trên đường RTP).
- Đây là “**pipeline latency**”, không phải “**glass-to-glass**”. Số 7,8 ms không bao gồm thời gian phơi sáng của cảm biến camera, đệm của màn hình, và mạng thật. Không so sánh được với số đo bằng cách quay hai màn hình.
- **MAVLink còn hẹp về tính năng:** chưa có giao thức nhiệm vụ/waypoint, chưa có geofence, và **takeoff** mới leo rồi giữ độ cao chứ chưa có trình tự hạ cánh tại điểm hay bay về nhà (RTL).

XII.2. Những điều đã làm sai rồi sửa

- “**Cờ DISCONT là dấu vân tay của mất gói.**” Sai. Một chú thích như vậy nằm trong receiver.cpp suốt một ngày, sống sót vì nó cho ra con số (0 trên link sạch) đúng như mong đợi — rồi vẫn cho ra 0 khi hai phần ba luồng không tới. *Một bộ đếm đồng ý với bạn chưa phải là bằng chứng; phải cho nó gặp trường hợp mà lẽ ra nó phải nhúc nhích.*

- “Ảnh tĩnh giấu được thiệt hại.” Sai hai lần. Lần chạy lại “thêm chuyển động” thực ra đã thay một ảnh tĩnh này bằng một ảnh tĩnh khác (pinwheel *không* hề chuyển động — 0,0% điểm ảnh đổi). Kết quả đổi không phải vì chuyển động.
- “Hai lần chạy cùng pipeline là so sánh được.” Sai — x264enc không tái lập được từng bit (mã hoá cùng đầu vào hai lần cho hai file khác nhau). Cả thí nghiệm điểm ảnh phải “đóng băng” bitstream trước, nếu không đã cho ra kết luận sai một cách tự tin.

XIII. Kết luận và hướng phát triển

Project đã làm được gì. Xây dựng và *đo đạc* một đường liên kết drone–mặt đất hai nửa: (1) đường truyền hình ảnh H.264/RTP/UDP hạ độ trễ từ 2288 ms xuống 7,8 ms và định lượng chính xác cơ chế lan truyền lỗi khi mất gói; (2) cổng MAVLink có watchdog failsafe 3 giây, telemetry quy đổi SI, và lệnh arm/takeoff/land được xác nhận, chạy end-to-end trên ArduPilot SITL thật; và (3) một dashboard thời gian thực gộp cả hai. Điểm khác biệt không nằm ở chỗ “chạy được” mà ở chỗ *mọi con số đều đo được, tái lập được, và có script kèm theo.*

Bài học lớn nhất. Bộ giải mã video có thể *che giấu* mất gói khiến phần mềm tưởng mọi thứ vẫn ổn; muốn biết sự thật phải đo ở đúng tầng (RTP), và phải đối chiếu mọi bộ đếm với một nguồn độc lập trước khi tin nó.

Hướng phát triển.

- Mô phỏng mất gói theo *chùm* (netem loss dạng Gilbert-Elliott) để tiến gần radio thật hơn.
- Thêm cơ chế chống lỗi ở tầng RTP: FEC (sửa lỗi tiến) hoặc RTX (truyền lại chọn lọc), rồi đo lại đánh đổi độ trễ.
- Giảm `key-int-max` có điều kiện / bitrate thích ứng để hồi phục nhanh hơn khi link xấu.
- Chạy trên Jetson Orin NX thật với bộ mã hoá phần cứng NVENC và đo lại glass-to-glass.
- Mở rộng MAVLink: giao thức nhiệm vụ, geofence, trình tự hạ cánh/RTL.

Tóm lại, đây không phải một thử nghiệm chạy một lần, mà là một hệ thống chạy được, có dữ liệu, có pipeline, có đo lường độc lập, và còn nhiều hướng để phát triển tiếp.

XIV. Tài liệu tham khảo

- [1] H. Schulzrinne et al., “RTP: A transport protocol for real-time applications”, Internet Engineering Task Force (IETF), RFC 3550, 2003. Accessed: Jul. 10, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3550>
- [2] Y.-K. Wang et al., “RTP payload format for H.264 video”, Internet Engineering Task Force (IETF), RFC 6184, 2011. Accessed: Jul. 10, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6184>
- [3] ITU-T, *H.264: Advanced video coding for generic audiovisual services*, ITU-T Recommendation H.264 (ISO/IEC 14496-10), 2021. Accessed: Jul. 10, 2026. [Online]. Available: <https://www.itu.int/rec/T-REC-H.264>
- [4] The GStreamer Team, *GStreamer: Open source multimedia framework — application development manual*, 2024. Accessed: Jul. 10, 2026. [Online]. Available: <https://gstreamer.freedesktop.org/documentation/>
- [5] MAVLink Development Team, *MAVLink developer guide*, 2024. Accessed: Jul. 10, 2026. [Online]. Available: <https://mavlink.io/en/>
- [6] ArduPilot Development Team, *ArduPilot sitl (software in the loop) simulator*, 2024. Accessed: Jul. 10, 2026. [Online]. Available: <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>
- [7] The Linux Foundation, *tc-netem(8): Network emulator — linux manual page*, 2023. Accessed: Jul. 10, 2026. [Online]. Available: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>
- [8] NVIDIA Corporation, *Jetson linux multimedia api — accelerated GStreamer*, 2024. Accessed: Jul. 10, 2026. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r36.3/DeveloperGuide/SD/Multimedia/AcceleratedGstreamer.html>
- [9] WHATWG, *HTML living standard — server-sent events*, 2024. Accessed: Jul. 10, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/server-sent-events.html>

Phụ lục

1. **AIVIETNAM:** Thông tin về AIVIETNAM có thể đọc thêm tại [đây](#).
2. **GitHub Repository:** github.com/kaisye/drone-video-link.
3. **Sổ tay khái niệm:** docs/04-CONCEPTS.md — giải thích mọi khái niệm từ đầu (nguồn của phần 4 báo cáo này).
4. **Số liệu độ trễ:** video/results/latency.md — bảng đo và cách tái lập.
5. **Số liệu mất gói:** video/results/packet-loss.md — ảnh sạch/vỡ/hồi phục và phương pháp.
6. **Demo:** docs/assets/demo.mp4 — 30 giây video nhận được, hỏng giữa chừng rồi hồi phục.